Ex 5: Analysis of Prompting Techniques in LLM Reasoning

Learning Objective:

Develop a Python program to analyze how different prompting strategies—Zero-shot, Few-shot, and Chain-of-Thought (CoT)—affect the reasoning capabilities of Large Language Models (LLMs). The program will also identify patterns in logical fallacies or hallucinations using benchmark datasets.

Steps:

1. Import Required Libraries: OpenAI API or Hugging Face Transformers or Google Gemini API and Additional libraries: datasets, pandas, matplotlib

2. Configure API Keys

- Set your OpenAI and Gemini API keys securely (do not share).

3. Load Dataset

- Use either **SQuAD** (QA reasoning) or **GSM8K** (math reasoning).

4. Initialize Models

5. Define Prompting Strategies

- **Zero-shot Prompt, Few shot prompt, Chain-of-Thought Prompt**

6. Create Functions to Query Models

7. Generate Outputs

- Apply all prompting techniques on the same dataset questions.

8. Display Results of Zero-shot Prompt, Few shot prompt, Chain-of-Thought Prompt

Program Code:

```
import re, time
import pandas as pd
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import pipeline
import google.generativeai as genai
from getpass import getpass
from google.colab import userdata

GEMINI_API_KEY = userdata.get('GEMINI_API_KEY')
genai.configure(api_key=GEMINI_API_KEY)
data = load_dataset("gsm8k", "main")["test"].select(range(2))
```

```
hf = pipeline("text-generation", model="distilgpt2")
gm = genai.GenerativeModel("gemini-2.5-flash")

def zero(q): return f"Q: {q}\nA:"
def few(q): return f"Q: 2+3?\nA: 5\nQ: 10-4?\nA: 6\nQ: {q}\nA:"
def cot(q): return f"Solve step by step.\nQ: {q}\nA:"

def ask_hf(p):
  try:
    return hf(p, max_new_tokens=40, do_sample=False,
pad_token_id=50256)[0]["generated_text"]
  except:
    return "HF Error"

def ask_gm(p):
  try:
    return gm.generate_content(p).text
  except:
    return "Quota exceeded / Try later"

def get_num(x):
  n = re.findall(r"-?\d+\.?\d*", str(x).replace(",", ""))
  return n[-1] if n else None

def get_gold(x):
     m = re.search(r"####\s*(-?\d+\.?\d*)", x.replace(",", ""))
     return m.group(1) if m else get_num(x) # Result

def check(pred, gold):
    if pred is None:
      return "Hallucination"
    elif pred == gold:
      return "Correct"
```

```
    else:
      return "Wrong"

res = []
for r in data:
  q = r["question"]
  gold = get_gold(r["answer"])

  for name, fn in [("Zero-shot", zero), ("Few-shot", few), ("CoT", cot)]:
    p = fn(q)

    hf_out = ask_hf(p)
    gm_out = ask_gm(p)
    time.sleep(10)

    hf_pred = get_num(hf_out)
    gm_pred = get_num(gm_out)

    res.append([name, q[:50], gold, hf_pred, gm_pred, check(hf_pred, gold), check(gm_pred,
gold)])

df = pd.DataFrame(res, columns=["Prompt", "Question", "Gold", "HF_Pred",
"Gemini_Pred", "HF_Result", "Gemini_Result"])
print(df)

acc = pd.DataFrame({ "HF": df["HF_Result"].eq("Correct").groupby(df["Prompt"]).mean() *
100, "Gemini": df["Gemini_Result"].eq("Correct").groupby(df["Prompt"]).mean() * 100 })
print("\nAccuracy (%)")
print(acc)

print("\nHF Summary")
print(df["HF_Result"].value_counts())
print("\nGemini Summary")
print(df["Gemini_Result"].value_counts())
```

Output:

	Prompt	Question	Gold	HF_Pred	Gemini_Pred	HF_Result	Gemini_Result
0	Zero-shot	Janet’s ducks lay 16 eggs per day. She eats th...	18	2	18	Wrong	Correct
1	Few-shot	Janet’s ducks lay 16 eggs per day. She eats th...	18	5	18	Wrong	Correct
2	CoT	Janet’s ducks lay 16 eggs per day. She eats th...	18	2	None	Wrong	Hallucination
3	Zero-shot	A robe takes 2 bolts of blue fiber and half th...	3	1.5	None	Wrong	Hallucination
4	Few-shot	A robe takes 2 bolts of blue fiber and half th...	3	-4	3	Wrong	Correct
5	CoT	A robe takes 2 bolts of blue fiber and half th...	3	1.5	None	Wrong	Hallucination

Accuracy (%)

Prompt	HF	Gemini
CoT	0.0	0.0
Few-shot	0.0	100.0
Zero-shot	0.0	50.0

```
HF Summary
HF_Result
Wrong    6
Name: count, dtype: int64

Gemini Summary
Gemini_Result
Correct          3
Hallucination    3
Name: count, dtype: int64
```

Learning Outcome:

Students will:

- Understand how Zero-shot, Few-shot, and CoT prompting influence LLM reasoning.
- Identify patterns in hallucinations and logical fallacies.
- Gain experience working with benchmark datasets like GSM8K or SQuAD.